Practica Laboratorio 2: Quicksort y Mergesort

Universidad de Carabobo
Facultad de Ciencias y Tecnología
Departamento de Computación

Integrantes:
Freddy Ovalle – C.I: 31.709.650
Juan Bermúdez – C.I: 31.608.369

Agosto de 2026

1 ¿Qué diferencias existen entre registros temporales ($t0–$t9) y registros guardados ($s0–$s7) y cómo se aplicó esta distinción en la práctica?

- Registros temporales ($t): Sus valores no se preservan a través de llamadas a subrutinas. En el código, se usaron para cálculos inmediatos y efímeros, como el cálculo de desplazamientos de memoria con sll y add (ej. sll $t3, $t2, 2), condiciones booleanas con slt y para cargar valores temporales desde la memoria (lw $t4, 0($t3)).

- Registros guardados ($s): Sus valores deben conservarse si una subrutina los modifica. En ambos algoritmos, al ser recursivos, variables críticas como low, high, mid y los contadores de los bucles se asignaron a $s0–$s5. Para cumplir la convención, estos registros se respaldaron en la pila (sw $s0, 0($sp)) al inicio de funciones como mergesort y partition, y se restauraron antes del jr $ra.

2 ¿Qué diferencias existen entre los registros $a0–$a3, $v0–$v1, $ra y cómo se aplicó esta distinción en la práctica?

- $a0-$a3 (Argumentos): Se usaron para pasar parámetros a las subrutinas. Por ejemplo, en merge, se usaron cuatro argumentos: dirección base ($a0), left ($a1), mid ($a2) y right ($a3). También se pasaron a syscall para imprimir texto y enteros.

- $v0-$v1 (Valores de retorno): Se utilizaron para devolver resultados de funciones. En partition, se retornó el índice del pivote usando move $v0, $s2. También se usaron para indicar los códigos de servicio de los syscall (ej. li $v0, 4 para strings).

- $ra (Return Address): Almacena la dirección de retorno tras ejecutar un jal. Dado que quicksort y mergesort son recursivas, $ra se guardó obligatoriamente en la pila en cada llamada para no perder la ruta de retorno a las funciones padre.

3 ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

El uso de registros es considerablemente más rápido que acceder a la memoria (instrucciones lw/sw). En los códigos proporcionados, las variables de iteración e índices (como i, j, k) se mantuvieron estrictamente en registros ($s y $t) dentro de los bucles loop_partition y merge_loop. La memoria solo se accedió cuando fue estrictamente necesario: para leer/escribir los elementos del arreglo y para gestionar la pila.

4 ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Las instrucciones de salto condicional e incondicional (beq, bne, j, jal) permiten la lógica de los algoritmos pero introducen sobrecarga. En arquitecturas segmentadas (pipelining), los saltos pueden causar riesgos de control"(control hazards), lo que obliga a vaciar el pipeline si la predicción de salto falla. Las llamadas recursivas también suman impacto debido a la constante escritura/lectura en la pila (overhead de memoria).

5 ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno MIPS32?

- Quicksort: Tiene una complejidad promedio de O(n log n) y en el peor caso de O(n^2). Es un algoritmo in-place (no requiere memoria extra más allá de la pila). En MIPS32, esto es muy eficiente porque minimiza el uso del bus de memoria de datos.

- Mergesort: Tiene una complejidad garantizada de O(n log n) en todos los casos, pero requiere memoria auxiliar. En el código, esto se evidencia en el uso del arreglo .space 40 (temp). En MIPS32, esto implica el doble de operaciones de escritura/lectura en memoria durante la fase copy_back, haciéndolo ligeramente más lento en constantes de tiempo debido al cuello de botella de la memoria.

6 ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

- Búsqueda (IF): Se extrae la instrucción de la memoria de instrucciones usando el Program Counter (PC).
- Decodificación (ID): Se interpreta la instrucción y se leen los registros implicados.
- Ejecución (EX): La ALU realiza operaciones matemáticas, lógicas o calcula direcciones de memoria (ej. offset + base).
- Memoria (MEM): Se accede a la memoria de datos (solo en lw y sw).
- Escritura (WB): Se guarda el resultado en el registro destino (si aplica).

7 ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

- Se utilizaron predominantemente instrucciones tipo I (addi, lw, sw, beq, bne, slti) porque el ordenamiento requiere constantes accesos a memoria (Load/Store), iteraciones con contadores inmediatos y saltos condicionales basados en comparaciones.
- Las instrucciones tipo R (add, sll, slt) fueron fundamentales para el cálculo dinámico de direcciones de memoria (ej. multiplicar por 4 el índice con sll para alineación de words).
- Las tipo J (j, jal) fueron esenciales para la recursividad y los saltos incondicionales de los bucles.

8 ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

Abusar de j, beq o bne interrumpe el flujo secuencial del programa. En un procesador real, esto genera "burbujas.en el pipeline, desperdiciando ciclos de reloj mientras se calcula la nueva dirección del contador de programa (PC), lo que degrada severamente el rendimiento general.

9 ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo Load/Store de MIPS obliga a que solo lw y sw accedan a la memoria, lo que permite que el resto de las operaciones se ejecuten a máxima velocidad en los registros internos. Además, el formato uniforme de instrucciones simplifica el diseño

del hardware, haciendo que las operaciones matemáticas de los algoritmos sean muy predecibles en ciclos de reloj.

10 ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

Esta herramienta permite rastrear la recursividad evaluando instrucción por instrucción. En el código, fue vital para observar cómo se decrementaba el $sp (Stack Pointer) al entrar a múltiples llamadas de quicksort o mergesort, y cómo se restauraba correctamente al hacer los jr $ra.

11 ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

El panel de "Registers" es fundamental para verificar la correcta preservación de los registros $s y los argumentos $a. Asimismo, el "Data Segment.[es] indispensable para observar en tiempo real cómo los valores de los arreglos array y temp se intercambian y sobrescriben en memoria durante los procesos de partition y merge.

12 ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

En MARS, para un add, la instrucción pasa por IF, ID (se leen dos registros de la Data Path), EX (la ALU realiza la suma de ambos valores). La fase MEM se ignora (no se toca la memoria de datos), y pasa directamente a WB, donde el resultado se escribe en el registro destino.

13 ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

Para un lw, tras IF e ID, la fase EX usa la ALU para sumar el registro base más el valor inmediato (extensión de signo) para calcular la dirección. En la fase MEM, esa dirección se usa para leer el valor en la memoria de datos. Finalmente, en WB, el dato extraído de la memoria viaja al banco de registros para guardarse.

14 Justificar la elección del algoritmo alternativo

Dado que la práctica exige ambos, se puede justificar el contraste: Quicksort se prioriza en entornos de memoria limitada (microcontroladores) por no requerir arreglos auxiliares, mientras que Mergesort se implementa cuando se requiere estabilidad y un rendimiento O(n log n) determinista y predecible, asumiendo que se dispone del espacio extra (temp).

15 Alisis y Discusión de los Resultados

Ambas implementaciones demuestran con éxito la traslación de algoritmos de alto nivel a código ensamblador MIPS32. El mayor reto resuelto en ambos códigos es la gestión del Stack Frame. Al ser algoritmos basados en "Divide y Vencerás", la recursividad exige guardar múltiples estados simultáneos. En Mergesort, se reservaron hasta 28 bytes en la pila por llamada (addi $sp, $sp, -28), frente a los 16/24 de Quicksort, demostrando que, a nivel de bajo nivel, la complejidad de un algoritmo no solo se mide en tiempo, sino en el manejo meticuloso de la memoria y la preservación del estado de la CPU.

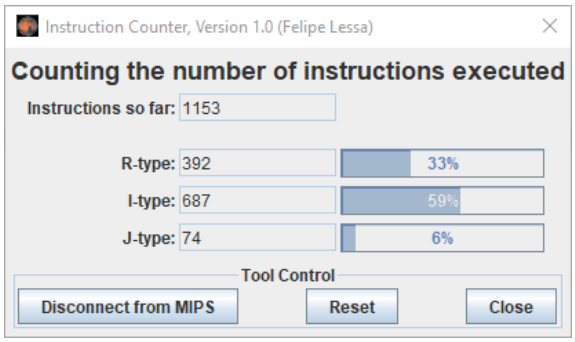


Figura 1: Resultados Quicksort.

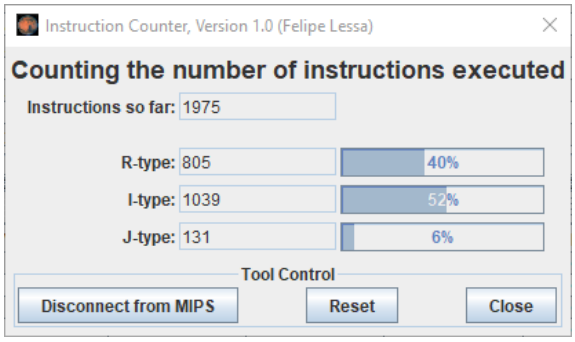


Figura 2: Resultados Mergesort.